



“La inteligencia detrás de tu productividad”.

MANUAL TÉCNICO GUÍA DE INSTALACIÓN WISE AGEND

Nombre del sistema	Wise Agend
Tipo de documento	Manual Técnico - Guía de Instalación
Versión del sistema	1.0
Fecha de elaboración	2026-01-13
Autor(es)	

Institución / empresa (si aplica): _____





“La inteligencia detrás de tu productividad”.

Indice

1. INTRODUCCIÓN.....	4
2. OBJETIVOS DEL DOCUMENTO.....	4
2.1 Objetivo General	4
2.2 Objetivos Específicos	4
3. ALCANCE DEL DOCUMENTO	4
4. REQUISITOS DEL SISTEMA	4
4.1 Requisitos de Hardware	4
4.2 Requisitos de Software.....	5
5. SOFTWARE NECESARIO.....	5
6. INSTALACIÓN DEL SOFTWARE	5
6.1 Instalación de Python (si se ejecuta el backend local).....	5
6.2 Instalación del gestor de paquetes / dependencias	6
6.3 Instalación del sistema gestor de base de datos	6
6.4 Instalación de otras dependencias.....	6
7. CONFIGURACIÓN INICIAL DEL ENTORNO	7
7.1 Variables y rutas	7
7.2 Configuración de Firebase.....	7
7.3 Configuración del endpoint del API.....	8
8. INSTALACIÓN DEL PROYECTO WISE AGEND	8
8.1 Clonado del repositorio	8
8.2 Estructura general del proyecto (referencia)	8
8.3 Instalación de dependencias del proyecto	8
8.4 Preparación de archivos de configuración.....	9
9. VERIFICACIÓN DE LA INSTALACIÓN	9
9.1 Ejecución en local	9
9.2 Pruebas básicas	9
9.3 Salida esperada	9
10. PROBLEMAS COMUNES Y SOLUCIONES.....	9
11. CONSIDERACIONES FINALES	10
12. GLOSARIO	10





“La inteligencia detrás de tu productividad”.

13. REFERENCIAS	10
-----------------------	----





“La inteligencia detrás de tu productividad”.

1. INTRODUCCIÓN

Esta guía describe el proceso de instalación y configuración mínima necesaria para ejecutar el sistema Wise Agend en un entorno local de desarrollo o pruebas. El sistema corresponde a una aplicación móvil desarrollada en Flutter que utiliza Firebase para autenticación y consume un API HTTP para la gestión de recordatorios (reminders) y secciones.

- Propósito: permitir que un usuario técnico configure el entorno y ejecute la aplicación sin intervención adicional.
- Dirigido a: desarrolladores, personal técnico y evaluadores que requieran compilar y ejecutar la app en Android/Windows/macOS.
- Relación con otros manuales: complementa documentos de arquitectura, despliegue/mantenimiento del backend y manual de usuario.

2. OBJETIVOS DEL DOCUMENTO

2.1 Objetivo General

Describir el proceso de instalación del entorno necesario para ejecutar el sistema Wise Agend.

2.2 Objetivos Específicos

- Identificar los requisitos del sistema.
- Guiar la instalación del software base (Flutter, herramientas de desarrollo y dependencias).
- Verificar la correcta instalación del entorno y la ejecución de la aplicación.

3. ALCANCE DEL DOCUMENTO

Esta guía cubre la instalación para entornos de desarrollo o pruebas en sistemas Windows 10/11, macOS y distribuciones Linux comunes. Incluye la configuración mínima para ejecutar la aplicación Flutter y conectarla a un backend disponible en red.

- Cubre: instalación de herramientas, descarga de dependencias, configuración de Firebase en la app y parametrización del endpoint del API.
- No cubre: despliegue productivo, hardening, CI/CD, publicación en tiendas (Google Play / App Store) ni configuración avanzada de infraestructura.

4. REQUISITOS DEL SISTEMA

4.1 Requisitos de Hardware

- Procesador: 2 núcleos o superior (recomendado 4 núcleos).
- Memoria RAM: mínimo 8 GB (recomendado 16 GB si se usa emulador).
- Espacio en disco: mínimo 10 GB libres (SDKs, cachés y dependencias).





“La inteligencia detrás de tu productividad”.

4.2 Requisitos de Software

- Sistema operativo compatible: Windows 10/11, macOS 12+ o Linux (Ubuntu 20.04+ recomendado).
- Acceso a internet para descargar SDKs y paquetes (pub.dev, Android SDK, etc.).
- Permisos para instalar software y configurar variables de entorno.

5. SOFTWARE NECESARIO

A continuación se listan las herramientas necesarias y su propósito dentro del sistema.

Software	Versión recomendada	Propósito
Flutter SDK (incluye Dart)	Flutter Stable (3.x)	Compilación y ejecución de la aplicación Flutter.
Android Studio + Android SDK	Última estable	Emulador y herramientas de compilación para Android.
Visual Studio Code (opcional)	Última estable	Edición de código y ejecución/depuración.
Git	2.4+	Clonado y control de versiones del repositorio.
Java JDK (para Android)	JDK 17 (recomendado)	Requerido por herramientas de Android/Gradle.
Python (backend, si aplica)	3.10+	Ejecución del API local de recordatorios (Flask).

Dependencias principales de la app (Flutter packages): firebase_core, firebase_auth, http, intl, flutter_local_notifications y timezone.

6. INSTALACIÓN DEL SOFTWARE

6.1 Instalación de Python (si se ejecuta el backend local)

Python se utiliza para ejecutar el servicio backend (API) que expone endpoints HTTP consumidos por la aplicación. Si el backend ya está desplegado en otra máquina/servidor, esta sección puede omitirse.

- Descargar desde el sitio oficial: <https://www.python.org/downloads/> (o gestor del sistema operativo).
- En Windows: durante la instalación marcar la opción 'Add Python to PATH'.
- En macOS: instalar con Homebrew (brew install python) o instalador oficial.
- En Linux (Ubuntu): sudo apt update && sudo apt install python3 python3-pip

Verificación:





“La inteligencia detrás de tu productividad”.

```
python --version
```

```
pip --version
```

6.2 Instalación del gestor de paquetes / dependencias

Para Flutter, el gestor de paquetes es 'pub' (incluido en Flutter). Para el backend en Python se utiliza 'pip'.

- Flutter: flutter --version (valida que Flutter y Dart estén instalados).
- Flutter: flutter doctor (valida dependencias del entorno).
- Python: pip install -r requirements.txt (si existe archivo de requerimientos del backend).

6.3 Instalación del sistema gestor de base de datos

Wise Agend utiliza servicios de Firebase (por ejemplo, autenticación) y puede apoyarse en almacenamiento administrado en la nube (p. ej., Firestore) a través del backend. Por lo tanto, no se requiere instalar un motor de base de datos local para ejecutar el entorno de pruebas, salvo que el backend del proyecto lo indique.

6.4 Instalación de otras dependencias

6.5 Instalación y configuración de Flutter (paso a paso)

Esta sección describe de forma detallada cómo instalar Flutter y dejar el entorno listo para compilar y ejecutar Wise Agend en un equipo con Windows.

Paso 1. Descargar Flutter SDK (<https://docs.flutter.dev/install>)

- 1) Descargar Flutter (canal Stable) desde la documentación oficial.
- 2) Descomprimir el SDK en una ruta recomendada (sin espacios), por ejemplo: C:\src\flutter

Paso 2. Configurar variable de entorno PATH

- 1) Abrir 'Editar las variables de entorno del sistema' en Windows.
- 2) En Variables del sistema, seleccionar 'Path' y presionar 'Editar'.
- 3) Agregar: C:\src\flutter\bin
- 4) Guardar cambios y reiniciar la terminal o el IDE.

Paso 3. Verificación inicial de Flutter

Ejecutar en terminal:

- flutter --version
- flutter doctor

El comando flutter doctor indica qué componentes hacen falta (Android SDK, licencias, etc.).

Paso 4. Instalar Android Studio y Android SDK

- 1) Instalar Android Studio.





“La inteligencia detrás de tu productividad”.

- 2) En 'SDK Manager' instalar:
 - Android SDK Platform (una versión reciente).
 - Android SDK Build-Tools.
 - Android SDK Platform Tools.
- 3) (Opcional) Crear un emulador en 'AVD Manager'.

Paso 5. Aceptar licencias de Android
Ejecutar: flutter doctor --android-licenses

Paso 6. Abrir el proyecto Wise Agend e instalar dependencias

- 1) Abrir la carpeta del proyecto (donde está pubspec.yaml).
- 2) Ejecutar: flutter pub get

Paso 7. Ejecutar la aplicación
Con un emulador/dispositivo conectado, ejecutar: flutter run

Paso 8. Configuración recomendada del endpoint del backend
Si se utiliza emulador Android y el backend se ejecuta en la misma computadora, utilizar:
http://10.0.2.2:5000
Si se utiliza un dispositivo físico, configurar la IP local del servidor backend, por ejemplo:
http://192.168.X.X:5000
Verificar que el firewall permita el puerto 5000 y que ambos equipos estén en la misma red.

- Instalar Flutter SDK y agregarlo al PATH (flutter/bin).
- Instalar Android Studio y descargar Android SDK, Platform Tools y un emulador.
- Aceptar licencias: flutter doctor --android-licenses

7. CONFIGURACIÓN INICIAL DEL ENTORNO

Esta sección describe la configuración mínima para que el proyecto pueda ejecutarse y comunicarse con sus servicios.

7.1 Variables y rutas

- Asegurar que 'flutter' esté disponible en terminal (PATH configurado).
- Android: configurar ANDROID_SDK_ROOT (si aplica) y validar con flutter doctor.

7.2 Configuración de Firebase

La aplicación inicializa Firebase en el arranque (Firebase.initializeApp()). Para compilar correctamente, debe existir una configuración de Firebase por plataforma.

- Android: agregar el archivo google-services.json en android/app/ y configurar el plugin de Google Services en Gradle.
- iOS: agregar GoogleService-Info.plist en Runner/ y ejecutar pod install (si se usa iOS).
- Habilitar Firebase Authentication (Email/Password) en el proyecto Firebase.





“La inteligencia detrás de tu productividad”.

7.3 Configuración del endpoint del API

El endpoint del backend se define en el código como baseUrl/baseHost. Por defecto se usa una dirección local de red.

- Archivo api.dart: baseUrl = "http://192.168.2.134:5000".
- Archivo recordatorios_service.dart: baseHost = "http://192.168.2.134:5000".
- Si se ejecuta en emulador Android y el backend está en la misma PC, puede requerirse usar http://10.0.2.2:5000.
- Asegurar que el dispositivo/emulador tenga conectividad hacia la IP/puerto del backend y que el firewall lo permita.

8. INSTALACIÓN DEL PROYECTO WISE AGEND

8.1 Clonado del repositorio

En una terminal, ejecutar:

```
git clone <URL_DEL_REPOSITORIO>  
cd <CARPETA_DEL_PROYECTO>
```

8.2 Estructura general del proyecto (referencia)

- lib/main.dart: punto de entrada de la aplicación (inicializa Firebase, intl y notificaciones locales).
- lib/api.dart: helper para autenticación con Firebase y consumo de endpoint /reminders.
- lib/services/recordatorios_service.dart: CRUD de recordatorios y secciones contra el backend.
- lib/services/notification_service.dart: programación y cancelación de notificaciones locales.

8.3 Instalación de dependencias del proyecto

En la raíz del proyecto:

```
flutter pub get
```

Para validar el entorno:

```
flutter doctor
```





“La inteligencia detrás de tu productividad”.

8.4 Preparación de archivos de configuración

- Configurar Firebase por plataforma (ver sección 7.2).
- Revisar el endpoint baseUrl/baseHost (ver sección 7.3).
- En Android 13+ verificar permisos de notificaciones (la app solicita permisos mediante flutter_local_notifications).

9. VERIFICACIÓN DE LA INSTALACIÓN

9.1 Ejecución en local

Con un emulador o dispositivo conectado:

```
flutter run
```

9.2 Pruebas básicas

- Inicio de sesión con usuario registrado en Firebase Authentication.
- Carga de recordatorios desde el endpoint /reminders (posible filtro por tipo: ?tipo=personal).
- Creación/edición/eliminación de recordatorios (según UI del proyecto).
- Programación de notificaciones locales para recordatorios próximos.

9.3 Salida esperada

- La aplicación muestra la pantalla de login y permite autenticarse sin errores.
- Al autenticarse, las peticiones HTTP incluyen el header Authorization: Bearer <ID_TOKEN>.
- Los recordatorios se listan correctamente (HTTP 200) y las notificaciones se muestran en el dispositivo en la fecha/hora programada.

10. PROBLEMAS COMUNES Y SOLUCIONES

- flutter doctor muestra faltantes: instalar Android SDK/accept licenses (flutter doctor --android-licenses).
- Error de Firebase (Missing google-services.json): agregar archivos de configuración por plataforma y limpiar (flutter clean).
- HTTP 401/403 en /reminders: validar que el usuario exista en Firebase y que el backend valide ID Token correctamente.
- No hay conexión al backend: confirmar IP/puerto, usar 10.0.2.2 en emulador, revisar firewall y que el backend esté ejecutándose.
- Notificaciones no aparecen: conceder permisos de notificaciones, revisar modo 'No molestar' y que se haya ejecutado NotificationService.init().





“La inteligencia detrás de tu productividad”.

11. CONSIDERACIONES FINALES

Para mantener reproducible el entorno de instalación, se recomienda fijar versiones de Flutter/Dart y de paquetes en pubspec.yaml, así como documentar el procedimiento de despliegue del backend y sus dependencias. Antes de realizar pruebas con dispositivos físicos, verificar permisos de red y notificaciones.

12. GLOSARIO

Término	Definición
Flutter	Framework de UI de Google para crear apps multiplataforma.
Dart	Lenguaje de programación utilizado por Flutter.
Firebase	Plataforma de Google para servicios backend (auth, base de datos, etc.).
ID Token	Token JWT emitido por Firebase Authentication para autenticar requests.
API	Interfaz HTTP que expone endpoints (/reminders, /sections) consumidos por la app.
Notificación local	Notificación generada por el dispositivo sin requerir push remoto.

13. REFERENCIAS

- Flutter - Documentación oficial: <https://docs.flutter.dev/>
- Firebase - Documentación oficial: <https://firebase.google.com/docs>
- flutter_local_notifications - Paquete: https://pub.dev/packages/flutter_local_notifications
- timezone - Paquete: <https://pub.dev/packages/timezone>
- Python - Documentación oficial: <https://docs.python.org/3/>





WISE AGEND

Ingeniería en Sistemas Computacionales
Tecnológico de Estudios Superiores de Jilotepec

“La inteligencia detrás de tu productividad”.

